

2021 最后题目内容

xml

graph.

THE UNIVERSITY OF AUCKLAND

Semester One 2023
Campus: City

COMPSCI 752

Big Data Management

(Time Allowed: 2 hours and additional time as specified)

NOTE

- The final exam is OPEN BOOK.
- Attempt ALL questions in this final exam.
- There are 100 marks in this exam.
- The final exam counts 60% towards your final mark.
- You need to submit your solutions as a single pdf file.
- We recommend you write down and draw your solutions by hand, and scan them in.

1. XPath

[15 marks]

Consider the tree in Figure 1. The document root is labelled by **D**, and element nodes are labelled by **E**. The tags of a node are indicated by **doc**, **a** or **b**. The context node for Question 1(c) is marked in pink.

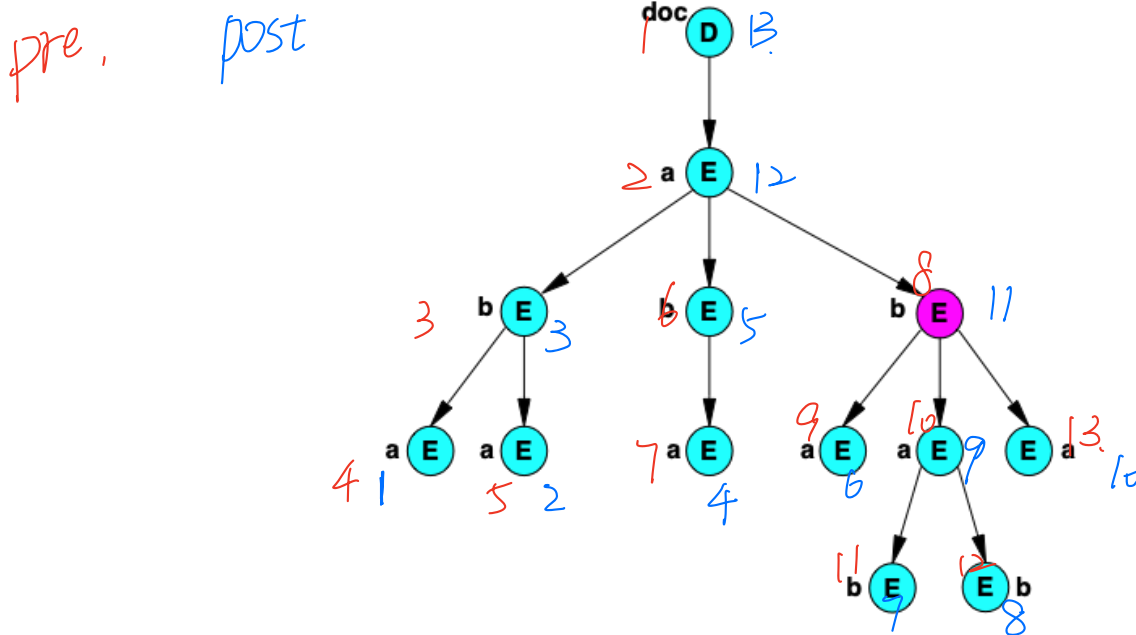


Figure 1: XML Tree for Question 1

- (a) For each node of the tree, write down the corresponding pre- and post-identifier.

[3 marks]

- (b) Write down the XML document that corresponds to this tree.

[3 marks]

- (c) With respect to the given context node, marked in pink, evaluate the following axes on the tree. List the pre-identifier of each node that each axis returns.

i. following::a

ii. ancestor-or-self::node()

iii. preceding-sibling::b

[3 marks]

- (d) Convert each of the following queries into the corresponding XPath expression, and evaluate it on the tree by listing the pre-identifier of the nodes returned.

- i. For every element child m of some element node, and for every node n that is the third among all **b**-labelled element children of m , return the node that is the second among all element children for some descendant-or-self node of n .

- ii. Return every element node whose number of preceding siblings is the same as its number of following siblings.

- iii. Return every element node that is not a leaf.

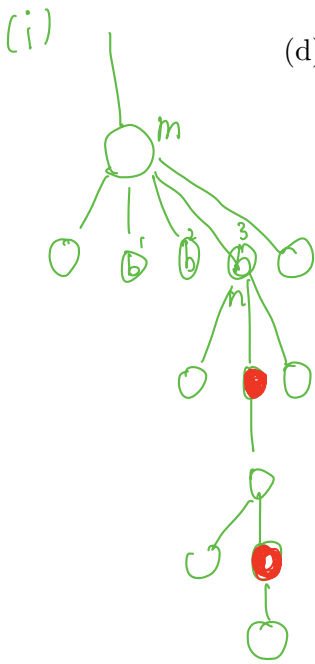
[6 marks]

$//*[not(*)]$

Page 1 of 7

4, 5, 7, 9, 11, 12, 13,

$//*[*]$ ← not a leaf



2. XQuery

[15 marks]

Consider the following XML document.

```
<?xml version="1.0"?>
<walks>
  <trek name="Kepler">
    <location>Fiordland</location>
    <difficulty>moderate</difficulty>
    <length>60km</length>
    <rating>88</rating>
  </trek>
  <trek name="Routeburn">
    <location>Fiordland</location>
    <difficulty>moderate</difficulty>
    <length>32km</length>
    <rating>98</rating>
  </trek>
  <trek name="Tongariro Crossing">
    <location>Tongariro</location>
    <difficulty>hard</difficulty>
    <length>19.4km</length>
    <rating>96</rating>
  </trek>
</walks>
```

Questions: Write down the following English language queries in XQuery, **and** show the result of evaluating the XQuery query on the given XML document.

- (a) For every distinct location l for which no trek with rating less than 85 exists, return an element node with label *location* that has the following content: i) a child element labelled *location_name* with the name of the location as text content, and ii) for each trek in that location, an element child labelled *trek* with the name of the trek as text content. [5 marks]

- (b) For the longest treks among those with difficulty **moderate**, return an element node labelled *trek* with i) attribute child *name* that contains the name of the trek and ii) an element child labelled *rating* that has the text of the rating of the trek as content. [5 marks]

- (c) For every distinct difficulty, return an element node *difficulty* with i) an attribute child called *value* that lists the difficulty, ii) an element child *trek* for every trek with that difficulty, and iii) an element child *avg_length* that contains the average length over all of these treks. [5 marks]

3. Graph databases

[20 marks]

Consider a simple application where walking treks are cycles (it ends where it starts) made up of sections between two huts, each offering a number #beds of beds and, potentially, showers. The sections have a distance in kilometers, and treks have a name and location.

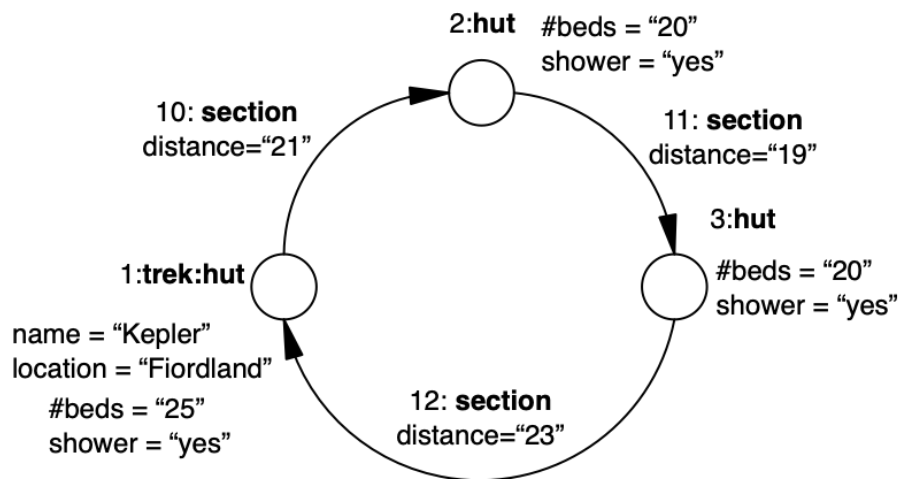


Figure 2: Property Graph for Question 3

Questions:

- Write down the formal property graph model $(V, E, \eta, \lambda, \nu)$ for the property graph in Figure 2. [4 marks]
- Write the following English language query in regular property graph logic:
Return the trek nodes that have 2 or 3 sections. [3 marks]
- Write the following English language query in regular property graph logic:
Find all pairs of hut nodes, each having showers and at least 15 beds, and directly connected by a section. [2 marks]
- Write the following English language query in regular property graph logic:
Return all trek nodes that have some section, each at most 25kms long and only connecting huts with showers available. [3 marks]
- Write down the queries from 3(b), 3(c), and 3(d) in regular property graph algebra. [8 marks]

4. PageRank and its MapReduce version

[15 marks]

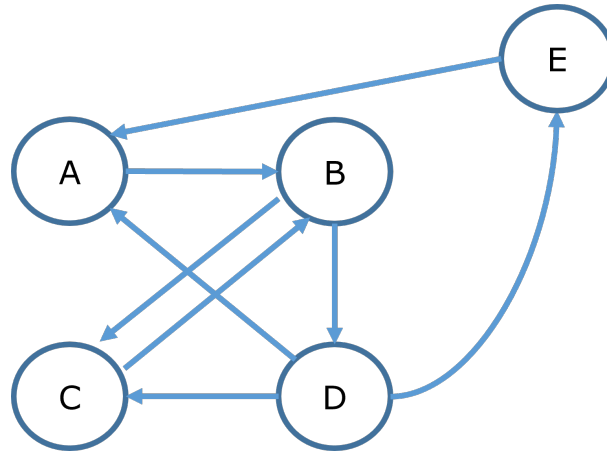


Figure 3: Graph 1 (for Questions 4(a), 4(b), 4(c))

We use Graph 1 in Figure 3 for executing the PageRank algorithm. For all questions, we use the damping factor $d = 1$. We only consider **one** iteration of the PageRank computation. The MapReduce version of PageRank is identical to the algorithm from the lecture, and is provided as follows.

Algorithm 1 `map(key: [url, pagerank], value: outlink_list)`

```

1: for each outlink in outlink_list do
2:   emit(key: outlink, value: pagerank/size(outlink_list))
3: end for
4: emit(key: url, value: outlink_list)

```

Algorithm 2 `reduce(key: url, value: list_pagerank_or_urls)`

```

1: outlink_list = [ ]
2: pagerank = 0
3: for each pagerank_or_urls in list_pagerank_or_urls do
4:   if is_list(pagerank_or_urls) then
5:     outlink_list = pagerank_or_urls
6:   else
7:     pagerank += pagerank_or_urls
8:   end if
9: end for
10: pagerank = d * pagerank + (1 - d) / n
11: emit(key: [url, pagerank], value: outlink_list)

```

Questions:

- (a) Compute the transition matrix $\mathbf{M}$ for Graph 1. [2.5 marks]
- (b) Given the initialized page rank vector $\mathbf{u}_0 = \{1/5, 1/5, 1/5, 1/5, 1/5\}$, compute the PageRank of each vertex after **one** iteration. [2.5 marks]
- (c) Given the initialized page rank vector $\mathbf{u}_0 = \{1/5, 1/5, 1/5, 1/5, 1/5\}$, illustrate just one iteration of the MapReduce-based version of PageRank in order to verify the previous result. For both Map and Reduce phases, show their inputs and outputs, respectively. [10 marks]

5. Prefix-filtering - MapReduce version

[15 marks]

We use the MapReduce version of the prefix-filtering algorithm to find all pairs of documents whose inner product similarity is at least a threshold $thres$. Assume that we know the maximum vector m in advance, and use it to compute the signature of each document. For each document i , we maintain d_i containing (1) the document index i , and (2) a vector containing all terms and its corresponding weight. The MapReduce version of prefix-filtering algorithm is identical to the algorithm from the lecture, and is provided as follows.

Algorithm 3 map(key: i , value: d_i)

```

1: Construct the prefix-filtering signature  $S(d_i)$ 
2: for each term  $t$  in the signature  $S(d_i)$  do
3:   emit(key:  $t$ , value:  $d_i$ )
4: end for

```

Algorithm 4 reduce(key: t , value: $D = [d_0, \dots, d_k]$, the list of k documents whose signature contains term t)

```

1: for each document pair  $\langle i, j \rangle$  where  $0 \leq i < j \leq k$  in the list  $D$  do
2:   Compute the similarity  $\sigma(d_i, d_j)$ 
3:   if  $\sigma(d_i, d_j) \geq thres$  then
4:     emit(key:  $\langle i, j \rangle$ , value:  $\sigma(d_i, d_j)$ )
5:   end if
6: end for

```

- (a) Explain the condition where MapReduce prefix-filtering runs faster than the standard prefix-filtering algorithm. [5 marks]
- (b) Explain the condition where MapReduce prefix-filtering runs as slow as the standard prefix-filtering algorithm. Present your solution to fix it. [10 marks]

6. Data stream - Bloom filter [10 marks]

Suppose we have n bits of memory available and a set S has m elements. Instead of using k hash functions each mapping an element to a bit in the memory, we could divide the n bits into k subarrays (assume n is divisible by k), and then use the i -th hash function, $i \in [1..k]$, to the i -th subarray.

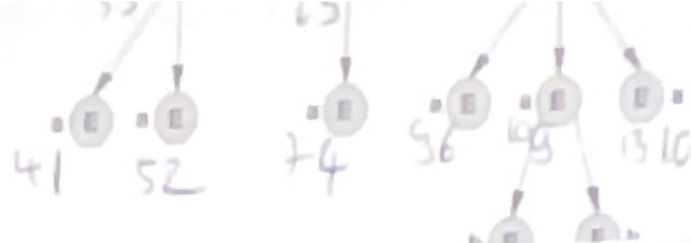
- (a) As a function of n , m , and k , what is the probability of a false positive? [5 marks]
- (b) How does it compare with the standard setting of Bloom filter that uses k hash functions into a single array? [5 marks]

7. Differential privacy [10 marks]

Consider the query: $f(D)$ = average weight of the people in the database D .

Assume that the weight of any person in D ranges between 50 and 100 kg. Consider a mechanism obtained by adding Laplacian noise $Lap(b)$ to the answer of the query, i.e. releasing $f(D) + Lap(b)$ where b is a parameter.

- (a) Under the assumption that D has 100 people, whose weight in range $[50, 100]$, does the mechanism above satisfy ϵ -differential privacy for some ϵ ? If the answer is yes, give the minimum such ϵ . If the answer is no, give a counterexample. [5 marks]
- (b) Under the assumption that D has **at least** 100 people whose weight in range $[50, 100]$, does the mechanism above satisfy ϵ -differential privacy for some ϵ ? If the answer is yes, give the minimum such ϵ . If the answer is no, give a counterexample. [5 marks]



b) $\langle ?_{xml} \text{ version} = "1.0" ? \rangle$

$\langle a \rangle$ or $\langle a / \rangle$

$\langle b \rangle$ $\langle a \rangle \langle /a \rangle$

$\langle a \rangle \langle /a \rangle$

$\langle /b \rangle$

$\langle b \rangle$

$\langle a / \rangle$

$\langle b \rangle$

$\langle b \rangle$

$\langle a / \rangle$

$\langle a \rangle$

$\langle b / \rangle$

$\langle b / \rangle$

$\langle /a \rangle$

$\langle a / \rangle$

$\langle b \rangle$

$\langle /a \rangle$

[a)

for \$l in distinct-values(\$location)
where every \$r in \$trek [\$location=\$l]/rating
satisfies (\$r ≥ 85)

return

<location>

<location_name>{\$l} <location_name

{ for \$t in \$trek [\$location=\$l]

return <trek>{\$t/data/@name} } </trek>

</location>

Out put:

<xml>

看回放 --

(b) let $\$w := \max(\| \text{tree}[\text{difficulty} = \text{"moderate"}] / \text{length})$
for $\$t$ in $\| \text{tree}[\text{difficulty} = \text{"moderate"}]$ and
length = $\$w$

return

<tree~~*~~ name = "{\\$t/data[@name]}">
<rating> \\$t/rating </rating>

</tree>

Out put.

Output

<tree~~*~~ name = "Planet Kepler">
<rating> 88 </rating>

</tree>

1. 10 $\mapsto$ (1, 2)
11 $\mapsto$ (2, 3)
12 $\mapsto$ (3, 1)

2. 1 $\mapsto$ {tree, hut}
2 $\mapsto$ {hut}
3 $\mapsto$ {hut}

10 $\mapsto$ {section}
11 $\mapsto$ {section}
12 $\mapsto$ {section}

3. (1, name) $\mapsto$ "Kyles"
(1, location) $\mapsto$ "Fionnland"
(1, #beds) $\mapsto$ 25
(1, shower) $\mapsto$ "Yes"

(2, #beds) $\mapsto$ 20
(2, shower) $\mapsto$ "Yes"

(3, #beds) $\mapsto$ 20
(3, shower) $\mapsto$ "Yes"

(10, distance) $\mapsto$ 21
(11, distance) $\mapsto$ 19
(12, distance) $\mapsto$ 23

$$b) \text{ result}(x) \leftarrow \text{trek}(x), \text{isection}(x, y), \text{isection}(y, x)$$

$$\text{hnt}(x), \text{hnt}(y)$$

$$\text{result}(x) \leftarrow \text{trek}(x), \text{isection}(x, y), \text{isection}(y, z)$$

$$\text{isection}(z, x)$$

$$\text{hnt}(x), \text{hnt}(y), \text{hnt}(z)$$

c) result $H(x, y) \leftarrow$ hu $h(x), : h(y), : section(x, y)$

$x, shower = \text{"Yes"}, y, shower = \text{"Yes"}$

$x, \#beds \geq 15, y, \#beds \geq 15$

d) reach(x,y) $\leftarrow$: section(x,y) ^{ASK} : lat(x) : lat(y)
x.shower = "Yes", y.shower = "Yes"

k.distance ≤ 25

result(x) $\leftarrow$ trek(x), ~~reach(x,y)~~
reach(x,y), reach*(y,x)

mean

≥ 0

≥ 1

$$(2) : \mathcal{M}_{\text{src}} \left(\overset{\phi}{\mathcal{M}}^{\text{I}}(\text{section}, \text{section})_{\text{src}_1, \text{trg}_2} \right) \cup \overset{\psi}{\mathcal{M}}(\text{section}, \text{section set})_{\text{src}_1, \text{trg}_3}$$

$$\phi : \text{trg}_2 = \text{src}_1, \lambda(\text{src}_1) = \text{trg}_1$$

$$\lambda(\text{src}_1) = \text{hwt}, \lambda(\text{src}_2) = \text{hwt}$$

$$\psi : \text{trg}_1 = \text{src}_2, \text{trg}_2 = \text{src}_3, \text{trg}_3 = \text{src}_1$$

$$\lambda(\text{src}_1) = \text{trg}_1, \lambda(\text{src}_1) = \text{hwt}$$

$$\lambda(\text{src}_2) = \text{hwt}$$

$$\lambda(\text{src}_3) = \text{hwt}$$

ii)

Φ (section)
ser/trg

$\Phi: d(\text{ser}) = \text{hnt}, d(\text{trg}) = \text{hnt}$

x. shower = "yes", x. # beds ≥ 15
y. shower = "yes", y. # beds ≥ 15

$$(ii) \quad \mathcal{N}_{\text{ser}}^{\Phi} \left(\mathcal{N}_{\text{ser}, \text{try}}^{\Phi} (\text{section}) \left(\mathcal{N}_{\text{ser}, \text{try}}^{\Phi} (\text{section}) \right) \right)$$

$$\Phi: d(\text{src}) = \text{hint}, d(\text{try}) = \text{hint}$$

$$\text{src. show} = \text{"Yes"}, \text{try show} = \text{"No"}$$

$$\text{edge. distance} \leq 25$$

a)

$$M = \begin{matrix} & \begin{matrix} + & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 1/4 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1/2 & 0 & 1/3 & 0 \\ 0 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 0 \end{bmatrix} \end{matrix}$$

b)

$$\begin{matrix} \begin{matrix} 0 & 0 & 0 & 1/3 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1/2 & 0 & 1/3 & 0 \\ 0 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 0 \end{matrix} + \begin{matrix} \text{---} & \text{---} & \text{---} & \text{---} & \text{---} \\ & & & & \text{---} \\ & & & & \text{---} \\ & & & & \text{---} \\ & & & & \text{---} \end{matrix} \end{matrix} \begin{bmatrix} 1/5 \\ 1/5 \\ 1/5 \\ 1/5 \\ 1/5 \end{bmatrix} = \begin{bmatrix} 1/15 + 1/5 \\ 2/5 \\ 1/10 + 1/15 \\ 1/10 \\ 1/15 \end{bmatrix} = \begin{bmatrix} 2/30 \\ 12/30 \\ 5/30 \\ 2/30 \\ 2/30 \end{bmatrix} = 1$$

or
41

Sum = 1

c) Input 相同

$(A, \frac{1}{5}), [B]$
 $(B, \frac{1}{5}), [C, D]$
 $(C, \frac{1}{5}), [E]$
 $(D, \frac{1}{5}), [A, C, E]$
 $(E, \frac{1}{5}), [A]$

Map

$\frac{1}{5} \times \frac{1}{2}$

Output

$(B, \frac{1}{5}), (A, [B])$
 $(C, \frac{1}{10}), (D, \frac{1}{10}), (B, [C, D])$
 $(B, \frac{1}{5}), (C, [B])$
 $(A, \frac{1}{5}), (C, \frac{1}{5}), (E, \frac{1}{5}), (D, [A, C, E])$
 $(A, \frac{1}{5}), (E, [A])$

Reduce

Input

把所有同类聚合

Output

$$(A, \frac{1}{15}), (A, \frac{1}{5}), (A, [13]) \longrightarrow (A, \frac{4}{15}), (A, [13]).$$

~

~.